

## BUỔI 3

# Validation và xử lý lỗi

Spring Boot 4 • Dự án YOEDU

### Mục tiêu của buổi học

- Áp dụng @Valid và constraint annotation
- Chuẩn hóa lỗi bằng GlobalExceptionHandler
- Phân biệt lỗi 400, 404 và business error

### Đầu ra mong đợi

- API có kiểm soát dữ liệu đầu vào và trả lỗi nhất quán.
- Sinh viên nhìn rõ workflow trước khi nhìn code
- Kết thúc buổi có bài tập mở rộng để tự hoàn thiện

Trọng tâm của bộ slide: vấn đề cần làm • luồng xử lý • file code cần mở • bài tập cuối buổi

## Roadmap của buổi học

Roadmap

### 1. Mở file

Xem đúng file của buổi học trước khi code

### 2. Workflow

Hiểu yêu cầu và luồng dữ liệu trước khi viết logic

### 3. Demo API

Chạy request theo đúng thứ tự để thấy hệ thống hoạt động

### 4. Bài tập

Mở rộng từ demo để sinh viên tự hoàn thiện

### Câu hỏi

- Validation nên đặt ở entity hay DTO?
- Khi nào dùng BadRequestException, khi nào dùng NotFoundException?

### Lưu ý

- Chưa mở quá nhiều file cùng lúc
- Chốt workflow trước rồi mới đi vào code
- Luôn gắn API demo với một nhu cầu nghiệp vụ cụ thể

# Mở file nào trước?

[Files](#)

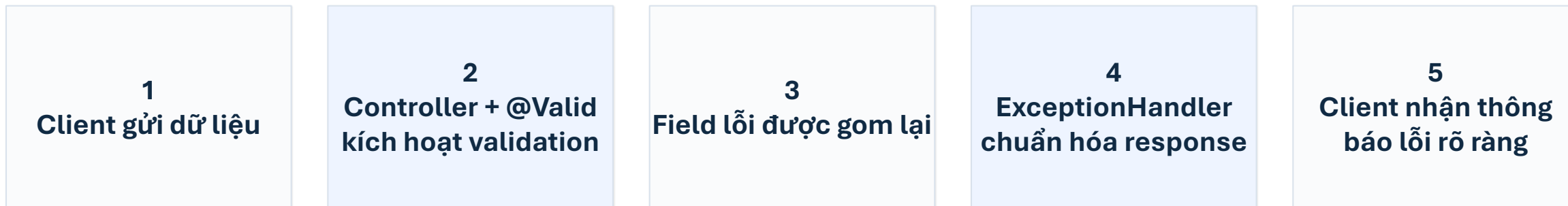
Chỉ mở các file lỗi của buổi học để sinh viên không bị ngợp bởi toàn bộ source.

| Thứ tự nên mở                 | Mục đích khi mở file            |
|-------------------------------|---------------------------------|
| 1 StudentUpsertRequest.java   | điểm đặt validation             |
| 2 GlobalExceptionHandler.java | gom lỗi về một nơi              |
| 3 BadRequestException.java    | mô tả lỗi nghiệp vụ             |
| 4 NotFoundException.java      | mô tả lỗi không tìm thấy        |
| 5 ApiResponse.java            | định dạng thành công / thất bại |

## Workflow validate request

Workflow A

Workflow nghiệp vụ nên giải thích trước khi mở từng method trong service.



Điểm chính:

Nhấn mạnh validation nằm ở DTO và lỗi được gom về một chuẩn chung để frontend dễ dùng.

## Workflow xử lý id không tồn tại

Workflow B

Workflow code / dữ liệu giúp sinh viên gắn bài toán với controller-service-repository.

1

Controller nhận id

2

Service gọi  
`repository.findById`

3

Không có dữ liệu

4

Ném  
`NotFoundException`

5

ExceptionHandler trả  
404 nhất quán

Chú ý:

Nhấn mạnh validation nằm ở DTO và lỗi được gom về một chuẩn chung để frontend dễ dùng.

## Những phần code quan trọng

Core code

Tập trung đúng entity, service, controller và endpoint cần dùng.

### Entity / enum

- Dùng lại Student / Parent

Entity mô tả dữ liệu

### Service

- StudentService (làm sạch logic lỗi)

Service giữ nghiệp vụ

### Controller

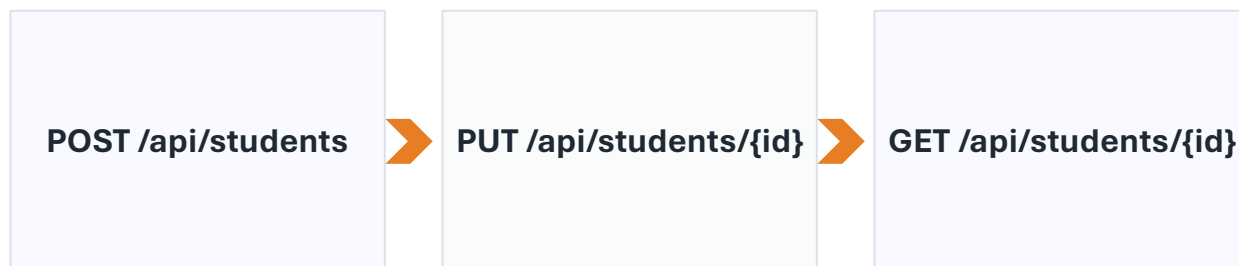
- StudentController

Controller mở API

## Chuỗi endpoint nên demo trên lớp

Endpoints

Nên chạy theo đúng thứ tự từ đầu vào → kết quả nhìn thấy rõ trên UI/Postman.



### Trình tự demo

- Chạy request thành công một lần trước
- Sau đó sửa dữ liệu để cho sinh viên thấy điểm thất bại
- Kết luận bằng dữ liệu trong DB / response DTO

### Điểm cần lưu ý

- Request body vào controller
- Method service nào đang xử lý
- Response nào frontend sẽ cần ở tuần React

# Checklist triển khai trong buổi

Checklist

### Những việc bắt buộc phải hoàn thành

- Validate ở DTO, không validate rả rác trong controller
- Thông báo lỗi phải đọc được
- Không lộ stack trace ra client
- Chuẩn response phải thống nhất cho toàn project



## Câu hỏi

Q&A

### Câu hỏi 1

- Validation nên đặt ở entity hay DTO?

### Câu hỏi 2

- Khi nào dùng `BadRequestException`, khi nào dùng `NotFoundException`?

### Câu hỏi 3

- Vì sao cần 1 format lỗi chung cho frontend?

# Bài tập cuối buổi và đầu ra mong đợi

Assignment

Slide này nên dùng để chốt buổi học và giao việc về nhà một cách rất cụ thể.

### Bài tập giao cuối buổi

- Thêm validation cho phone, fullName, status
- Bổ sung kiểm tra trùng mã học viên
- Tạo 3 request sai để quan sát response lỗi

### Sinh viên cần mang được gì sau buổi này?

- API có kiểm soát dữ liệu đầu vào và trả lỗi nhất quán.
- Hiểu được ít nhất 1 workflow đầy đủ
- Biết file nào là file lỗi để tiếp tục tự học

**Thông điệp chốt buổi: “Hiểu workflow trước, đọc đúng file trước, rồi mới code nhanh và đúng.”**